

ТЕХНИЧЕСКИ УНИВЕРСИТЕТ – СОФИЯ

**ФАКУЛТЕТ „КОМПЮТЪРНИ СИСТЕМИ И
ТЕХНОЛОГИИ“**



**ДВУПОСОЧНО ИНЖЕНЕРСТВО МЕЖДУ
ИЗХОДЕН КОД НА C++, UML ДИАГРАМИ НА
КЛАСОВЕ И РЕЛАЦИОННИ СХЕМИ НА
ДАННИ**

КУРСОВ ПРОЕКТ

по дисциплина „UML обектно-ориентирано проектиране“

Разработил:

Алекс Цветанов, фак. № **[TODO: фак. номер]**

Специалност: Компютърно и софтуерно инженерство, ОКС „Магистър“

Преподавател:

Проф. д-р инж. Даниела Гоцева

София, **[TODO: година на предаване]**

СЪДЪРЖАНИЕ

УВОД	1
I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД	3
II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА	5
III. ПРОЕКТИРАНЕ НА СИСТЕМАТА	7
IV. ПРОГРАМНА РЕАЛИЗАЦИЯ	9
V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА	11
VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ	13
ЗАКЛЮЧЕНИЕ	15
ЛИТЕРАТУРА	16
ПРИЛОЖЕНИЯ	18
Приложение А. Формат на вътрешния модел	18
Приложение Б. Примерен вход и получените диаграми	18
Приложение В. Породена релационна схема	18
Приложение Г. Изходен текст на програмата	18

УВОД

Обектно-ориентираният модел и програмният код, който го реализира, живеят в два различни носителя. Моделът се описва на UML и служи за разговор между хора, а кодът се описва на език за програмиране и служи за разговор с машина. Двата носителя се разминават почти веднага след като работата по проекта започне, защото кодът се променя всеки ден, а диаграмата се преначертава, когато някой се сети. Резултатът е познат: диаграма, на която вече никой не вярва, и код, чиято структура се възстановява само с четене.

Настоящият курсов проект разглежда това разминаване като инженерна задача с решение, а не като организационен навик. Ако структурата на кода може да бъде извлечена машинно, а UML моделът може да бъде записан в текстов вид, то преходът между двете посоки е преобразуване между два формални езика и подлежи на автоматизация. Терминологичната рамка за двете посоки е утвърдена от Chikofsky и Cross [1]: *обратно инженерство* е възстановяване на абстракция от реализация, а *право инженерство* е обратният преход от абстракция към реализация. Двете заедно образуват *двупосочно инженерство*, което е предметът на този проект.

Целта на проекта е да се проектира и реализира инструмент, наречен Blueprint, който извлича UML модел на класове от изходен код на C++, изобразява го в текстови нотации за диаграми, извежда от него релационна схема на данни и в обратната посока поражда заготовки на класове и схема по подаден модел, като допълнително отчита местата, на които моделът и кодът са се разминали.

За постигането на целта са формулирани следните **задачи**:

1. преглед на понятийния модел на UML, на градивните му елементи, на видовете връзки и на правилата и общите механизми, доколкото те засягат диаграмата на класове и нейното съответствие с изходен код;
2. анализ на съществуващите инструменти за извличане на структура от код и за двупосочно инженерство, и обосновка на избора на подход и на технологичен набор;
3. проектиране на вътрешния модел на системата, на границите между модулите и на правилата за преобразуване между клас, диаграма и релационна схема;
4. програмна реализация на извличането, на извеждането в двете текстови нотации, на пораждането на схема и заготовки и на проверката за разминаване;

5. изграждане на методика за експериментална проверка на верността на преобразуванията и на устойчивостта им при преминаване през пълен цикъл;
6. провеждане на експериментите и анализ на получените резултати.

Обхватът на работата е структурната част на обектно-ориентирания модел: класове, атрибути, операции, видимост, обхват, шаблонни класове и връзките между класовете, заедно със съответствието им на релационна схема. Диаграмите на случаите на употреба и на дейностите се разглеждат като контекст на модела и се използват за описание на самата система, но не се извличат от изходен код. Извън обхвата остават поведенческите диаграми, които изискват анализ на потока на управление, както и пълното семантично покритие на езика C++.

Структура на изложението. Раздел I излага предметната област и понятийния апарат на UML. Раздел II съпоставя възможните решения и обосновава избора. Раздел III представя проектирането на системата, а Раздел IV нейната програмна реализация. Раздел V описва методиката за проверка, а Раздел VI получените резултати. Заключението обобщава постигнатото и очертава по-нататъшната работа.

I. ПРЕДМЕТНА ОБЛАСТ И ЛИТЕРАТУРЕН ПРЕГЛЕД

UML е графичен език за визуализиране, специфициране, конструиране и документиране на артефактите на една програмна система. Той не е метод и не предписва процес на разработка, а речник и правила за съчетаване на думите от този речник. Тази разлика е съществена за настоящия проект: щом езикът е формален, а не илюстративен, той има недвусмислена абстрактна структура и следователно подлежи на машинна обработка. Официалната спецификация на езика [2] задава именно тази структура под формата на метамодел, а ръководствата на авторите на езика [3, 4] го излагат в приложим за проектант вид.

Понятийният модел на UML стъпва на три групи съставки. Първата са *градивните елементи*: структурни (клас, интерфейс, компонент, възел), поведенчески (взаимодействие, състояние) и групиращи (пакет), заедно с пояснителния елемент (бележка). Втората са *връзките*: зависимост, асоциация, обобщение и реализация, като асоциацията има отделни разновидности за агрегация и композиция, различаващи се по това дали частта може да съществува без цялото. Третата са *правилата и общите механизми*: спецификации, украшения, общи разделения и *механизмите за разширяване*, тоест ограничения, етикетираны стойности и стереотипи. Настоящият проект се интересува непосредствено от последните, защото именно чрез стереотип и етикетирана стойност се обозначава кой клас участва в релационната схема и как.

Диаграмата на класове е основната структурна диаграма и е носител на речника на системата. Класификаторът в нея се описва с име, атрибути, операции и отговорности, като видимостта и обхватът се отбелязват върху всеки член. Практиката за откриване на този речник е по-стара от самия UML: картите клас, отговорност и сътрудничество, познати като CRC карти, са предложени от Beck и Cunningham [5] точно като начин класовете да бъдат намерени преди да бъдат нарисувани. За компактно въведение в приложната част на езика е използван и [6].

Отношението между обектния модел и релационния модел на данните е самостоятелна тема със собствена литература. Релационната схема се описва чрез таблици, ключове и връзки между тях и има свой понятиен предшественик в модела същност и връзка на Chen [7]. Съответствието между клас и таблица не е взаимно еднозначно: наследяването няма пряк аналог в релационния модел, а асоциацията се изразява чрез

външен ключ или чрез свързваща таблица според кратността. Систематичен преглед на утвърдените стратегии за това съответствие дава [8]. Тези стратегии са непосредствената основа на правото инженерство в настоящия проект.

Мястото на двупосочното инженерство в жизнения цикъл на програмния продукт следва таксономията на Chikofsky и Cross [1]. Обратното инженерство поддържа документацията вярна, без да изисква дисциплина от разработчика, и е приложимо и към чужд код. Правото инженерство пренася решение, взето на нивото на модела, надолу до реализацията, без ръчно преписване. Стойността на двете заедно е в третото им следствие: щом и двете посоки са автоматизирани, разминаването между модел и код става измеримо, а не просто подозирано. **[TODO: при разширяване на раздела да се добави преглед на инструментите за проверка на съответствие между модел и код]**

II. АНАЛИЗ НА ВЪЗМОЖНИТЕ РЕШЕНИЯ И ОБОСНОВКА НА ИЗБОРА

Изборът на начин за прочитане на изходния текст на C++ предопределя горната граница на всичко останало. Разгледани са три възможности. Първата е анализ с регулярни изрази върху текста на файловете, който се пише за часове, но се проваля при шаблони, пространства от имена, макроси и всяко условно компилиране, тоест точно там, където реалният код е интересен. Втората е използване на съществуващ инструмент за документация, който вече разбира езика, и извличане на модела от неговия изход. Третата е ползване на самия компилаторен фронтенд през програмен интерфейс. Приет е третият подход чрез `libclang` [9], защото единствен той дава достъп до разрешените имена, типовете след заместване на шаблонните аргументи и точното местоположение на всяка декларация, а те са необходими, за да се различи агрегация от композиция.

Отделно е разгледан `Doxygen` с `Graphviz` [10, 11] като най-близкият утвърден инструмент. Той чете C++ и рисува диаграми на наследяване и на сътрудничество и по това покрива част от задачата. Три обстоятелства обаче го правят негоден за целта тук. Диаграмите му са изображения, предназначени за четене от човек, а не модел, върху който може да се разсъждава машинно. Той не различава разновидностите на асоциацията по семантика на собствеността, а само отбелязва наличието на връзка. И най-важното, той няма права посока: от неговия изход не се поражда нито код, нито схема. Тоест той решава едната половина от задачата и по устройство не може да реши другата.

Форматът на извеждане е втората ос. Възможностите са пряко рисуване във векторен формат, извеждане в стандартизирания XMI обмен за UML модели и извеждане в текстова нотация за диаграми. Прякото рисуване изисква реализация на алгоритъм за подреждане на графа и измества фокуса на проекта от моделирането към геометрията. XMI е точен и стандартен, но е нечетим за човек и е полезен само ако насрещният инструмент вече го приема. Избрано е извеждане в две текстови нотации, `PlantUML` [12] и `Mermaid`, защото двете са четими в необработен вид, версионират се като текст и се изобразяват от съществуващи изобразители. Двете, а не една, защото различията между тях са проверка, че вътрешният модел е независим от изходния формат.

Третата ос е как се описва участието на един клас в релационната схема. Разгледани са външен описателен файл, извеждане по съглашение за именуване и вграждане на

описанието в самия код. Външният файл се разминава с кода по същата причина, по която се разминава и диаграмата. Съгласението за именуване е тихо и се чути при първото изключение. Избрано е описанието да живее в кода чрез атрибути на C++20 [13], които на нивото на модела се изобразяват като UML стереотипи и етикетирани стойности. Така механизмът за разширяване на UML и механизмът за аотиране на C++ съвпадат по предназначение и преходът между тях е пряк.

Изборът на език за реализация следва от избора на фронтенд. C++20 е приет, защото libclang е библиотека на C и се обвива без междинен слой, защото инструментът работи върху C++ и разработчикът му остава в същата среда, и защото по-ранна работа на същия автор върху съпоставяне на класове и релационни схеми по време на компилация е решавала сродна задача от другата ѝ страна [14]. За система за компилиране е приет CMake, а за модулно тестване GoogleTest, като и двете са утвърдени и не изискват обосновка отвъд наличието им в екосистемата на езика.

III. ПРОЕКТИРАНЕ НА СИСТЕМАТА

Системата е разделена на пет модула с еднопосочни зависимости. *Четецът* обхожда единицата за превод и произвежда вътрешен модел. *Моделът* е независим от езика на входа и от формата на изхода и е единственото място, където се пази структурата. *Излъчвателите* превръщат модела в текст на избраната нотация. *Пораждащият модул* превръща модела в релационна схема и в заготовки на класове. *Проверяващият модул* съпоставя два модела и съобщава разликите между тях. Зависимостите сочат само към модела, така че нито един излъчвател не знае за съществуването на четеца. **[TODO: диаграма на пакетите с посоките на зависимостите]**

Речникът на системата е получен по метода на CRC картите [5]: за всяка предполагаема отговорност е записан носителят ѝ и сътрудниците му, след което картите без собствено поведение са отпаднали или са се слели със съседите си. Така са отделени класификаторите на модела. Всеки клас в модела носи име, квалифицирано с пространството от имена, списък от атрибути, списък от операции, вид на класификатора и списък от приложените върху него стереотипи и етикетирани стойности. Атрибутът носи име, тип, видимост, обхват и кратност. Операцията носи име, тип на резултата, списък от параметри, видимост и признаци за виртуалност и константност. Видимостта и обхватът се отбелязват по правилата на UML, като статичните членове се маркират като членове с обхват на класификатора, а не на екземпляра.

Връзките между класовете се извеждат от типовете на членовете, а не се обявяват отделно. Обобщението следва пряко от списъка на базовите класове. Асоциацията се установява, когато член на един клас се отнася към друг клас от модела. Разграничението между агрегация и композиция се прави по това дали обектът се притежава: член по стойност или член в притежаващ интелигентен указател дава композиция, защото частта загива заедно с цялото, докато член указател или отправка без притежание дава агрегация. Зависимостта се извежда от употребата на тип в подпис на операция без той да е член. Кратността се извежда от контейнерния тип, ако такъв присъства. Правилата са изнесени в таблица, за да бъдат ревизуеми поотделно. **[TODO: Таблица със съответствието тип на член към вид на връзката и кратност; препратка към нея в този абзац]**

Шаблонните класове се записват като параметризирани класификатори. Пълните специализации се записват като отделни класификатори със връзка на реализация

към общия шаблон, а частичните специализации се отбелязват с ограничение върху параметрите. Изборът е съзнателен: разгръщането на всяка инстанция в отделен клас затрупва диаграмата с еднотипни възли и заличава самата параметризация, която е носителят на смисъла. **[TODO: решение за представянето на инстанциите, породени от кода, а не от декларациите]**

Съответствието с релационната схема е описано с механизмите за разширяване на UML. Клас, обозначен със стереотип за постоянно съхранение, поражда таблица, всеки негов атрибут поражда колона, а етикетиранияте стойности задават име на таблицата, тип на колоната и участие в първичния ключ. Асоциацията с кратност едно към много поражда външен ключ в страната с много, а кратността много към много поражда свързваща таблица. За наследяването е приета една от утвърдените стратегии [8] и тя се избира с етиктирана стойност върху базовия клас, вместо да бъде зазидана в инструмента. **[TODO: коя стратегия за наследяване е приета по подразбиране]**

Използването на системата е описано с диаграма на случаите на употреба, а редът на операциите при преминаване през пълен цикъл с диаграма на дейностите. Двете диаграми са част от документацията на самия проект и служат едновременно като пример за нотацията, разгледана в Раздел I. **[TODO: Фигура с диаграма на случаите на употреба]**
[TODO: Фигура с диаграма на дейностите за пълния цикъл]

IV. ПРОГРАМНА РЕАЛИЗАЦИЯ

Реализацията е на C++20 [13] и се компилира с CMake. Проектът е разделен на библиотека, която съдържа цялата логика, и изпълним файл, който е тънък слой над нея за работа от команден ред. Разделението не е украса: модулните тестове се свързват срещу библиотеката и не минават през командния ред, така че проверката на поведението е независима от начина на извикване. Публичните заглавни файлове живеят в `include/`, реализацията в `src/`, а тестовете в `tests/`.

Четенето на изходния текст става през `libclang` [9]. Библиотеката дава дърво от курсори, което се обхожда рекурсивно, като за всеки курсор се проверява видът му и се решава дали той поражда елемент в модела. Ключово за верността на резултата е, че единицата за превод се отваря със същите ключове на компилатора, с които се компилира и самият проект, тоест с базата от команди за компилиране, произведена от CMake. Без нея пътищата към заглавните файлове се разминават, част от типовете остават неразрешени и моделът излиза непълен, при това мълчаливо. **[TODO: коментар на изходния текст на обхождането, с листинг от `src/`]**

Вътрешният модел е съвкупност от прости структури от данни без поведение, свързани по име, а не по указател. Изборът е продиктуван от изискването моделът да може да бъде записан и прочетен обратно, а също и да бъде сравняван с друг модел по стойност. Разрешаването на имената до връзки се извършва след обхождането, когато всички класове вече са известни, защото типът на член може да сочи към клас, деклариран по-късно в същата единица за превод.

Излъчвателите в двете нотации [12] споделят един обход на модела и се различават само по низовете, които произвеждат за всеки елемент. Общата част е изнесена в базов клас с виртуални методи за отделните елементи, а конкретните излъчватели предефинират само тях. Това е и мястото, където се вижда цената на разликите между нотациите: не всяка връзка в PlantUML има точен аналог в Mermaid, така че при част от елементите се налага приближение. **[TODO: списък на елементите, за които двете нотации се разминават, и приетото приближение за всеки]**

Пораждащата посока е реализирана като две отделни преобразувания над един и същ модел. Първото произвежда описание на релационна схема, а второто заготовки на класове. Заготовките се пораждат само за декларации, тоест инструментът не пише

тела на операции и не докосва съществуващ файл, който вече съдържа реализация. Провереният модул сравнява модел, извлечен от кода, с модел, прочетен от диаграма, и извежда разликите като списък от установени несъответствия. **[TODO: формат на съобщенията за несъответствие и кодовете за завършване на процеса]**

V. МЕТОДИКА ЗА ЕКСПЕРИМЕНТАЛНА ПРОВЕРКА

Проверката отговаря на три отделни въпроса и затова е разделена на три експеримента. Първият пита дали извлеченият модел отговаря на изходния текст. Вторият пита дали преминаването през пълен цикъл запазва модела. Третият пита дали проверяващият модул открива разминаване, което е внесено нарочно. Въпросите са различни и смесването им в едно число би скрило точно това, което трябва да бъде видяно.

Първият експеримент използва набор от малки единици за превод, всяка от които е съставена така, че да съдържа една езикова конструкция: наследяване, член по стойност, член в притежаващ указател, член отправка, шаблонен клас, частична специализация, влагане в пространство от имена и така нататък. За всяка единица очакваният модел е записан на ръка преди инструментът да бъде пуснат върху нея. Сравнението е между очакван и получен модел, а не между два изхода на самия инструмент, за да не се проверява инструментът сам срещу себе си. **[TODO: окончателен списък на конструкциите в набора и брой на единиците за превод]**

Вторият експеримент проверява устойчивостта при пълен цикъл. Изходен текст се превръща в модел, моделът в диаграма, диаграмата обратно в модел, и двата модела се сравняват по стойност. Ако преобразуванията са без загуба на информация, двата модела съвпадат. Този експеримент е по-силен от първия, защото не изисква очакван резултат да бъде написан на ръка и следователно може да бъде проведен върху истински код, а не само върху нарочно съставени примери. Като вход се предвижда самият изходен текст на проекта, което го прави и проверка на инструмента върху него самия. **[TODO: допълнителни проекти с отворен код за вход и критерий за подбора им]**

Третият експеримент внася разминаване нарочно. От вече съгласувана двойка код и диаграма се променя само едната страна: добавя се член, сменя се видимост, премахва се базов клас. След всяка промяна проверяващият модул се пуска и се записва дали и как е съобщил за нея. Оценяването е по два показателя, дял на откритите внесени разминавания и дял на съобщенията, които не отговарят на действително разминаване. Двата показателя се докладват поотделно, защото инструмент, който съобщава за всичко, постига идеална откриваемост и е безполезен.

Средата за провеждане на измерванията се записва изцяло, тъй като резултатите

зависят от версията на фронтенда: операционна система, версия на компилатора, версия на LLVM и libclang, версия на CMake и режим на построяване. Всяко измерване се извършва след чисто построяване и се повтаря, като разминаване между повторенията се разглежда като дефект в методиката, а не като шум. Формата на резултатите е фиксирана предварително и е описана в Раздел VI, за да не бъде избрана след като числата вече са известни. **[TODO: точните версии на средата, попълват се в деня на измерването]**

VI. ЕКСПЕРИМЕНТАЛНИ РЕЗУЛТАТИ И АНАЛИЗ

Резултатите се представят в реда на трите експеримента от Раздел V. Всяка таблица е придружена от коментар за това какво следва от нея и какво не следва. Разминаванията, които останат неразрешени, се изброяват поименно, а не се обобщават в дял, защото един необяснен случай е по-полезен от една закръглена стойност.

Първият експеримент проверява верността на извличането по отделни езикови конструкции. Резултатът се записва по един ред на конструкция, с изричен признак за съвпадение с предварително записания очакван модел (Табл. 1).

Таблица 1. Вярност на извличането по езикови конструкции

Конструкция	Съвпадение	Бележка
Наследяване	[TODO: ?]	[TODO: ?]
Член по стойност	[TODO: ?]	[TODO: ?]
Член в притежаващ указател	[TODO: ?]	[TODO: ?]
Член указател без притежание	[TODO: ?]	[TODO: ?]
Шаблонен клас	[TODO: ?]	[TODO: ?]
Частична специализация	[TODO: ?]	[TODO: ?]

[TODO: коментар на Таблица 1 след попълването ѝ: кои конструкции се извличат вярно, кои не, и защо]

Вторият експеримент проверява запазването на модела при пълен цикъл. Резултатът се записва по един ред на входен проект, поотделно за двете нотации, тъй като загубата на информация е свойство на нотацията, а не на модела (Табл. 2).

Таблица 2. Запазване на модела при пълен цикъл

Входен проект	Нотация	Съвпадение	Загубени елементи
Изходен текст на проекта	PlantUML	[TODO: ?]	[TODO: ?]
Изходен текст на проекта	Mermaid	[TODO: ?]	[TODO: ?]
[TODO: втори проект]	PlantUML	[TODO: ?]	[TODO: ?]
[TODO: втори проект]	Mermaid	[TODO: ?]	[TODO: ?]

[TODO: коментар на Таблица 2: кои елементи се губят при преминаване през всяка от двете нотации и дали загубата е поправима]

Третият експеримент проверява откриваемостта на нарочно внесено разминаване. Двата показателя се докладват поотделно, тъй като единият се подобрява за сметка на другия (Табл. 3).

Таблица 3. Откриваемост на нарочно внесено разминаване

Вид на промяната	Открита	Излишни съобщения
Добавен атрибут	[TODO: ?]	[TODO: ?]
Премахната операция	[TODO: ?]	[TODO: ?]
Сменена видимост	[TODO: ?]	[TODO: ?]
Премахнат базов клас	[TODO: ?]	[TODO: ?]
Сменена кратност	[TODO: ?]	[TODO: ?]

[TODO: коментар на Таблица 3 и обща инженерна трактовка на трите таблици заедно]

ЗАКЛЮЧЕНИЕ

В курсовия проект е проектиран и реализиран инструмент за двупосочно инженерство между изходен код на C++, UML диаграма на класове и релационна схема на данни. Приетият подход поставя в средата един вътрешен модел, независим както от езика на входа, така и от формата на изхода, и свежда всяко преобразуване до преход между този модел и една външна форма. Следствието е, че добавянето на нова нотация не засяга извличането, а промяна в извличането се отразява едновременно във всички изходни формати.

Предимствата на решението произтичат от избора на компилаторен фронтенд за прочитане на кода. Понятия като разрешено име, тип след заместване на шаблонните аргументи и собственост върху обект са достъпни точно защото анализът е семантичен, а не текстов. Това е и условието разграничението между агрегация и композиция да бъде правено по устройство, а не по предположение. Ограниченията са в същата посока: инструментът е толкова верен, колкото е верна подадената му база от команди за компилиране, и не вижда нищо, което препроцесорът е скрил при конкретния набор от ключове.

Резултатите от трите експеримента и тяхната инженерна трактовка се вписват тук след провеждането им. **[TODO: обобщение на измереното: вярност на извличането, запазване на модела при пълен цикъл, откриваемост на внесеното разминаване]**

За по-нататъшна работа се очертават четири посоки. Първата е разширяване на покритието на езика към конструкции, останали извън обхвата на проекта. Втората е свързване на проверката за разминаване с процеса на непрекъсната интеграция, така че отклонение между модела и кода да спира интеграцията, а не да чака някой да го забележи. Третата е допълване на модела с поведенческа информация, което е предпоставка за пораждаване на диаграми отвъд структурните. Четвъртата е обратният преход от съществуваща релационна схема към клас, който затваря кръга и от страната на данните.

ЛИТЕРАТУРА

- [1] E. J. Chikofsky and J. H. Cross, “Reverse engineering and design recovery: A taxonomy,” *IEEE Software*, vol. 7, no. 1, pp. 13–17, 1990.
- [2] Object Management Group, “OMG unified modeling language (OMG UML), version 2.5.1,” Tech. Rep. formal/2017-12-05, Object Management Group, 2017.
- [3] G. Booch, J. Rumbaugh, and I. Jacobson, *The Unified Modeling Language User Guide*. Addison-Wesley, 2 ed., 2005.
- [4] J. Rumbaugh, I. Jacobson, and G. Booch, *The Unified Modeling Language Reference Manual*. Addison-Wesley, 2 ed., 2004.
- [5] K. Beck and W. Cunningham, “A laboratory for teaching object-oriented thinking,” in *Conference Proceedings on Object-Oriented Programming Systems, Languages and Applications (OOPSLA '89)*, pp. 1–6, 1989.
- [6] M. Fowler, *UML Distilled: A Brief Guide to the Standard Object Modeling Language*. Addison-Wesley, 3 ed., 2003.
- [7] P. P.-S. Chen, “The entity-relationship model — toward a unified view of data,” *ACM Transactions on Database Systems*, vol. 1, no. 1, pp. 9–36, 1976.
- [8] S. W. Ambler, *Agile Database Techniques: Effective Strategies for the Agile Software Developer*. John Wiley and Sons, 2003.
- [9] LLVM Project, “Clang c interface to clang (libclang) — documentation.” https://clang.llvm.org/doxygen/group__CINDEX.html. Достъпен онлайн.
- [10] D. van Heesch, “Doxygen manual.” <https://www.doxygen.nl/manual/>. Достъпен онлайн.
- [11] J. Ellson, E. Gansner, L. Koutsofios, S. C. North, and G. Woodhull, “Graphviz — open source graph drawing tools,” in *Graph Drawing, LNCS 2265*, pp. 483–484, Springer, 2002.
- [12] A. Roques, “PlantUML language reference guide.” <https://plantuml.com/guide>. Достъпен онлайн.
- [13] ISO/IEC, “ISO/IEC 14882:2020, programming languages — C++,” tech. rep., International Organization for Standardization, 2020.

- [14] Алекс Цветанов, “[**TODO: точното заглавие на дипломната работа за ОКС „Бакалавър“**],” Дипломна работа за ОКС „Бакалавър“, Технически университет — София, Факултет „Компютърни системи и технологии“, [**TODO: година на защита**].

ПРИЛОЖЕНИЯ

Приложение А. Формат на вътрешния модел

Вътрешният модел е записваем във външен вид, за да може да бъде сравняван и съхраняван между отделните стъпки на пълния цикъл. **[TODO: описание на полетата на записа и листинг с дефиницията на структурите от `include/`]**

Приложение Б. Примерен вход и получените от него диаграми

Приложението съдържа малка единица за превод, обхващаща наследяване, агрегация и композиция, заедно с получените от нея текстове в двете нотации и изобразените диаграми. **[TODO: листинг с примерния вход] [TODO: получен текст в PlantUML] [TODO: получен текст в Mermaid] [TODO: Фигура с изобразената диаграма]**

Приложение В. Породена релационна схема

Приложението съдържа схемата, породена от анотираните класове на примера от Приложение Б. **[TODO: листинг с породената схема и коментар за приетата стратегия при наследяването]**

Приложение Г. Изходен текст на програмата

Пълният изходен текст се вмъква от `src/` и `include/` с `lstinputlisting`, за да не се разминава с действителния. **[TODO: списък на файловете, които се прилагат, и съответните команди за вмъкване]**